

What is it ?

- Algorithm to find shortest path in a weighted, non-negative graph
- greedy strategy

How to do it ?

- Select the unvisited node with the lowest distance, mark it as seen, and then update the distances of its neighboring nodes by comparing the current known distance with a potentially shorter path through the current node
- For each unvisited node, calculate the distance by adding the current node's distance to the edge weight. If this new distance is less than the previously known distance, update the distance and path
- My term of how to do it:
 - Setup 3 array: seen - to check visited, prev - to track path, dists - to track distance from the source, mark $\text{dists}[\text{source}] = 0$
 - Keep doing this while there is at least one unvisited node and that node is still reachable from source
 - Find the best node next to visit, lowest distance from dists array and has not been seen (called lo)
 - mark it seen
 - visit its neighbor, calculate neighbor distance ($\text{dists}[\text{lo}] + \text{edge weight}$)
 - If the distance is shorter, update the current dists of the neighbor as well as its prev
 - return path from prev array

Running time ?

- $O(V^2 + E)$ for adjacency list + linear scan for next node
- If switch to a min-heap/priority queue for "lowest unvisited", it becomes about: $O(\log V(V+E))$

Why heap can be used ?

- A minimum heap eliminates the need for a separate seen array, automatically tracking visited nodes and providing fast $O(\log V)$ node selection and update operations